

How the Agent System Is Built

Architecture, routing logic, prompt design, and structured actions behind the Supabase Edge Function that powers every agent in Auralith AI — with the actual source, end to end.

Runtime — Supabase Edge Function (Deno), fronted by a React/TypeScript client

Model — Groq API, openai/gpt-oss-120b

Storage — Supabase Postgres with row-level security

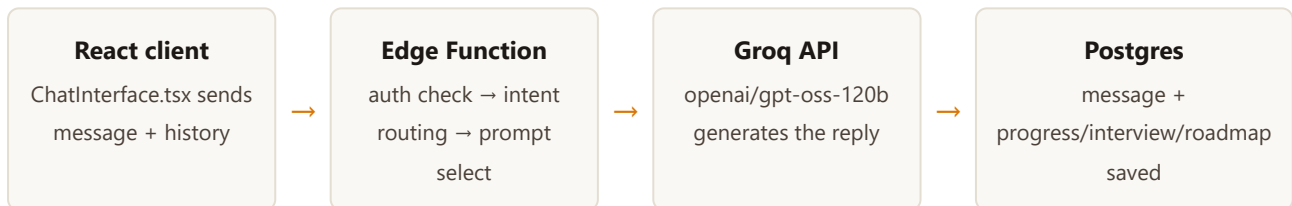
Scope — supabase/functions/ai-agent/index.ts and its client integration

Contents

01	Architecture Overview	request lifecycle, end to end
02	Data Model	tables and row-level security
03	Authenticating Every Request	why <code>--no-verify-jwt</code> plus manual auth
04	Intent Detection	auto-routing a message to an agent
05	Per-Agent System Prompts	one prompt template per specialist
06	Calling the Model	the Groq request/response contract
07	DSA Topic Detection	and the bug that shaped it
08	Structured Action: Interview Grading	turning a transcript into a score
09	Structured Action: Roadmap Generation	turning a conversation into a plan
10	Client Integration	React state machine around one fetch call
11	Security Notes	what's actually enforced, and where

01 Architecture Overview

Every agent in Auralith AI — Learning, DSA, Review, Debug, Interview, Roadmap, Progress, and General — is served by a **single** Supabase Edge Function, `ai-agent`. There is no per-agent backend; the function receives a message plus which agent (or none) the client thinks it should use, resolves that into a system prompt, and forwards the conversation to Groq.



Two more request shapes ride the same endpoint without a separate route: `action: "end_interview"` and `action: "generate_roadmap"`. Both skip the conversational system prompts entirely and instead ask the model to return strict JSON, which the function parses and the client persists as structured rows rather than chat text.

REQUEST SHAPE	WHAT IT TRIGGERS	RETURNS
<code>{ message, agent_type, history }</code>	Normal chat turn	<code>{ content, agent_type, metadata }</code>
<code>{ action: "end_interview", history }</code>	Grade the whole transcript	<code>{ result: InterviewGrade }</code>
<code>{ action: "generate_roadmap", history }</code>	Extract a structured plan	<code>{ result: RoadmapContent }</code>

02 Data Model

Everything the agents read or write lives in seven tables, all with row-level security scoped to `auth.uid()` so a user can only ever see their own rows — enforced at the database layer, independent of anything the client or edge function does.

TABLE	PURPOSE
<code>profiles</code>	Skill level, goals, preferred languages, GitHub/LeetCode handles
<code>chats</code>	One row per conversation, tagged with its agent type
<code>messages</code>	Every user/assistant turn, with per-message <code>metadata</code> jsonb
<code>progress_reports</code>	Score + problems_solved per DSA topic, seeded to 0 on signup
<code>interview_results</code>	Graded mock interviews: type, score, feedback, topics_covered
<code>roadmaps</code>	Structured plan in a <code>content</code> jsonb column
<code>documents</code>	Reserved for a future RAG knowledge base

supabase/migrations – RLS pattern applied to every table

```
ALTER TABLE progress_reports ENABLE ROW LEVEL SECURITY;

CREATE POLICY "select_own_progress" ON progress_reports FOR SELECT
  TO authenticated USING (auth.uid() = user_id);

CREATE POLICY "insert_own_progress" ON progress_reports FOR INSERT
  TO authenticated WITH CHECK (auth.uid() = user_id);
```

`roadmaps.content` is a single jsonb blob rather than a normalized weekly-plan table. That was a deliberate simplification: a roadmap is generated whole (by the model) and read whole (by the UI), so there was nothing to gain from splitting it into rows — and it means marking a week complete is a single `UPDATE` that merges one field, not a join.

03 Authenticating Every Request

The function is deployed with `--no-verify-jwt`, which sounds like it disables auth — it doesn't. Supabase's new-format publishable API keys (`sb_publishable_...`) aren't JWTs, so the platform's built-in gateway-level JWT check rejects every request outright if left on, including legitimate ones. Turning it off just moves the auth check *into* the function, where it's done properly against the actual logged-in user rather than a static key:

supabase/functions/ai-agent/index.ts

```
const authHeader = req.headers.get("Authorization") ?? "";
const token = authHeader.replace(/^Bearer\s+/i, "");

if (!token) {
  return new Response(
    JSON.stringify({ error: "Missing Authorization header" }),
    { status: 401, headers: { ...corsHeaders, "Content-Type": "application/json" } }
  );
}

const supabaseClient = createClient(SUPABASE_URL, SUPABASE_ANON_KEY);
const { data: { user }, error: authError } = await supabaseClient.auth.getUser(token);

if (authError || !user) {
  return new Response(
    JSON.stringify({ error: "Unauthorized" }),
    { status: 401, headers: { ...corsHeaders, "Content-Type": "application/json" } }
  );
}
```

The client sends the signed-in user's real session token, not the anon key, as the bearer credential — so `getUser(token)` is checking an actual person, not just "some request with the app's public key attached":

src/components/workspace/ChatInterface.tsx

```
const response = await fetch(
  `${import.meta.env.VITE_SUPABASE_URL}/functions/v1/ai-agent`,
  {
    method: 'POST',
    headers: {
      Authorization: `Bearer ${session?.access_token}`,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({ message: text, agent_type: currentAgentType, history }),
  }
)
```

VERIFIED

Calling the deployed function with no Authorization header, or with a garbage token, returns `401` in both cases; calling it with a real user's session token returns `200`. Confirmed against the live endpoint with throwaway test accounts before and after this change shipped.

04 Intent Detection

When the client doesn't pin a specific agent (or the user is on the "Auto" tab), the function classifies the message itself with an ordered stack of regexes. Order matters here — a message mentioning both "bug" and "algorithm" should route to Debug, not DSA, so the more specific/urgent categories are checked first and the catch-alls last:

supabase/functions/ai-agent/index.ts

```
function detectIntent(message: string): AgentType {
  const m = message.toLowerCase();

  if (/\\b(bug|error|fix|crash|exception|undefined|null|syntax|runtime|compile|
  traceback|stacktrace|not working|broken|issue|problem with my code)\\b/.test(m))
    return "debug";

  if (/\\b(review|refactor|optimize|clean|readability|best practice|code quality|
  smell|improve my (code|solution|function))\\b/.test(m))
    return "review";

  if (/\\b(interview|mock|hr question|behavioral|system design|oop interview|
  dbms interview|os interview|hire|placement)\\b/.test(m))
    return "interview";

  if (/\\b(roadmap|learning path|study plan|curriculum|how to learn|where to start|
  beginner to|internship prep|job ready|career path)\\b/.test(m))
    return "roadmap";

  if (/\\b(progress|weak|topic analysis|how am i doing|performance|score|report|
  dashboard|analytics)\\b/.test(m))
    return "progress";

  if (/\\b(leetcode|problem|solution|algorithm|solve|two sum|binary search|
  linked list|tree|graph|dp|dynamic programming|brute force|optimal|complexity|
  big o|time complexity|space complexity)\\b/.test(m))
    return "dsa";

  if (/\\b(explain|teach|what is|how does|concept|understand|learn|example|notes|
  definition|difference between|vs\\b)\\b/.test(m))
    return "learning";

  return "general";
}
```

This only ever runs when the incoming `agent_type` is `"general"` or missing — if the user has manually pinned an agent tab, that choice is respected as-is and the classifier never gets a vote.

05 Per-Agent System Prompts

Every agent shares one base instruction (always format with markdown, never hallucinate), then layers on a structural template that forces the model's response into a specific shape — this is what makes the DSA agent always walk brute-force → optimized → optimal instead of just answering however it feels like that turn:

supabase/functions/ai-agent/index.ts – shared base + one specialist

```
const base = `You are CodeMentor AI, a world-class AI coding mentor. Always format responses with clear structure using markdown: headings (**Title**), bullet points (- item), numbered lists, and fenced code blocks (\`\`\`language...\`\`\`). Keep explanations precise and practical. Never hallucinate – if unsure, say so.`;
```

```
dsa: `${base}
```

```
You are the DSA Agent. For every problem, follow this exact structure:
```

```
**Problem Understanding**
```

```
- Restate the problem clearly
```

```
**Brute Force Approach**
```

```
- Explain the naive solution with code
```

```
- Time/Space: O(?) / O(?)
```

```
**Optimized Approach**
```

```
- Explain the better solution with code
```

```
**Optimal Approach**
```

```
- Best possible solution with full code
```

```
- Dry run with example input
```

```
**Complexity Analysis**
```

```
- Time Complexity: O(?)
```

```
- Space Complexity: O(?)`,
```

AGENT	FORCES THE RESPONSE INTO
-------	--------------------------

Learning	Overview → key points → code example → pitfalls → practice problems
----------	---

DSA	Brute force → optimized → optimal, each with complexity
-----	---

Review	Strengths → issues → optimizations → rewritten code → score /10
--------	---

Debug	Error identified → root cause → fix → corrected code → prevention
-------	---

Interview	One question at a time, each answer scored /10 with a model answer
-----------	--

Roadmap	Current level → target → missing skills → weekly plan → resources
---------	---

Progress	Summary → strong topics → weak areas → focus → weekly goal
----------	--

General	Open-ended; detects what's needed and behaves like the right specialist
---------	---

All eight prompts (and the two structured-action prompts in sections 8 and 9) are plain template strings, selected with one lookup — there's no prompt-management system, because eight hand-written templates in one file is the simplest thing that could possibly work, and simplicity was worth more here than flexibility that wasn't needed yet.

06 Calling the Model

The function talks to Groq's OpenAI-compatible chat-completions endpoint. This is the one piece that was swapped mid-project — it originally called Gemini directly; Groq's free tier (1,000–14,000+ requests/day depending on model, versus Gemini's 20/day cap) made it the better fit for a hobby-tier deployment, and because the endpoint is OpenAI-shaped, the swap was a reshape of one function, not a rewrite of the app:

supabase/functions/ai-agent/index.ts

```
const GROQ_URL = "https://api.groq.com/openai/v1/chat/completions";
const GROQ_MODEL = "openai/gpt-oss-120b";

async function callGroq(
  systemPrompt: string,
  history: Array<{ role: string; content: string }>,
  userMessage: string
): Promise<string> {
  if (!GROQ_API_KEY) {
    return "**API Key Missing**\n\nThe `GROQ_API_KEY` secret is not configured...";
  }

  const messages = [
    { role: "system", content: systemPrompt },
    ...history.map((h) => ({ role: h.role, content: h.content })),
    { role: "user", content: userMessage },
  ];

  const res = await fetch(GROQ_URL, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${GROQ_API_KEY}`,
    },
    body: JSON.stringify({ model: GROQ_MODEL, messages, temperature: 0.7, max_tokens: 4096 }),
  });

  if (!res.ok) {
    const errText = await res.text();
    throw new Error(`Groq API error ${res.status}: ${errText.slice(0, 200)}`);
  }

  const data = await res.json();
  const text = data?.choices?.[0]?.message?.content;
  if (!text) throw new Error("Empty response from Groq API");
  return text;
}
```

Gemini's API shape needed a separate `systemInstruction` field and mapped the assistant role to `"model"`; Groq/OpenAI's shape is a flat `messages[]` array with `system` / `user` / `assistant` roles directly — that's the entire migration surface. Every system prompt, every piece of routing logic, and the whole rest of the app didn't need to change.

07 DSA Topic Detection

Whenever the resolved agent is `"dsa"`, the function also tries to tag which topic the exchange was about, so the client can nudge that topic's progress score. Ten fixed topics, one regex each, checked in a fixed priority order so a generic term like "array" doesn't swallow more specific ones like "dynamic programming":

supabase/functions/ai-agent/index.ts

```
const TOPIC_PATTERNS: Array<{ topic: string; pattern: RegExp }> = [
  { topic: "Dynamic Programming", pattern: /\b(dynamic programming|bdp|memoiz|
tabulation|knapsack|subsequence|subset sum)\b/ },
  { topic: "Binary Search", pattern: /\b(binary search|log n search|search.*sorted array)\b/ },
  { topic: "Linked Lists", pattern: /\b(linked list|listnode|singly linked|doubly linked|
\.next\b)\b/ },
  { topic: "Graphs", pattern: /\b(graph|adjacency|dijkstra|topological sort|
connected component|bipartite|union find)\b/ },
  { topic: "Trees", pattern: /\b(binary tree|btree|treenode|bbst|inorder|
preorder|postorder|leaf node)\b/ },
  { topic: "Hashing", pattern: /\b(hash ?map|hash ?set|hashing|hash table|
frequency map|dictionary lookup)\b/ },
  { topic: "Recursion", pattern: /\b(recursion|recursive|backtrack)\b/ },
  { topic: "Sorting", pattern: /\b(sort|quicksort|mergesort|bubble sort|insertion sort)\b/ },
  { topic: "Strings", pattern: /\b(substring|anagram|palindrome|string manipulation)\b/ },
  { topic: "Arrays", pattern: /\b(array|subarray|matrix|in-place|two pointer|
sliding window)\b/ },
];

function detectTopic(message: string, content: string): string | null {
  // Prefer the user's stated intent – DSA explanations often reference other
  // techniques in passing (e.g. an Arrays problem solved via a "dynamic
  // programming approach" like Kadane's algorithm), which makes scanning the
  // full response an unreliable signal on its own.
  return matchTopic(message.toLowerCase()) ?? matchTopic(content.toLowerCase());
}
```

WHY THE MESSAGE IS CHECKED FIRST, NOT THE RESPONSE

The first version of this scanned the model's *full generated answer* for topic keywords. Testing it against "give me an array question" returned `Dynamic Programming` — because the model solved it with Kadane's algorithm and explained its optimal solution as "a dynamic programming approach," which is a legitimately correct thing to say about that problem. Scanning the model's own explanation for topic hints turned out to be unreliable exactly because a good answer cross-references adjacent techniques. The fix was to check the user's stated intent first and only fall back to scanning the response when the message itself gives no signal at all (e.g. a generic "give me a hard problem").

08 Structured Action: Interview Grading

"End Interview" doesn't call a different model or a different endpoint — it sends the entire transcript back through `callGroq` with a grading-specific system prompt that demands JSON instead of markdown, then parses that JSON on the server before it ever reaches the client:

supabase/functions/ai-agent/index.ts

```
const INTERVIEW_GRADING_PROMPT = `You are an interview grading assistant. You will
receive the full transcript of a mock technical interview between an AI interviewer
and a candidate. Analyze the ENTIRE conversation and respond with ONLY a JSON object
– no markdown, no code fences, no explanation outside the JSON – in exactly this shape:
{
  "interview_type": "<one of: DSA, System Design, Behavioral, OOP, DBMS, OS, Mixed>",
  "score": <integer 0-100>,
  "feedback": "<2-4 sentence summary of strengths and weaknesses>",
  "topics_covered": ["<topic1>", "<topic2>"]
}
Base the score on correctness, clarity of communication, problem-solving approach, and
completeness of answers. If the candidate barely answered anything substantively, score
low (0-20) and say so in the feedback.`;

async function gradeInterview(history: Array<{ role: string; content: string }>) {
  const transcript = history.map((h) => `${h.role.toUpperCase()}: ${h.content}`).join("\n\n");
  const raw = await callGroq(INTERVIEW_GRADING_PROMPT, [], transcript);
  const parsed = extractJSON(raw) as Partial<InterviewGrade>;

  return {
    interview_type: typeof parsed.interview_type === "string" ? parsed.interview_type : "Mixed",
    score: Math.max(0, Math.min(100, Math.round(Number(parsed.score) || 0))),
    feedback: typeof parsed.feedback === "string" ? parsed.feedback : "No feedback generated.",
    topics_covered: Array.isArray(parsed.topics_covered) ? parsed.topics_covered.map(String) : [],
  };
}
```

Every field is defensively coerced and clamped on the way out — `score` is forced into `0-100` (matching the database's own `CHECK` constraint), and a missing or malformed field falls back to a safe default rather than propagating `undefined` into a `NOT NULL` column insert.

Parsing model output that's *supposed* to be pure JSON but sometimes isn't

```
function extractJSON(text: string): unknown {
  const trimmed = text.trim().replace(/^```(?:json)?/i, "").replace(/```$/, "").trim();
  try {
    return JSON.parse(trimmed);
  } catch {
    const match = trimmed.match(/\{[\s\S]*\}/);
    if (match) return JSON.parse(match[0]);
    throw new Error("Could not parse structured JSON from model response");
  }
}
```

The client then computes `duration_minutes` from the chat's first message timestamp to the moment "End Interview" is clicked, inserts the graded result into `interview_results`, and posts a formatted summary back into the chat so the transcript reads as a closed conversation rather than one that just stops.

09 Structured Action: Roadmap Generation

"Save as Roadmap" follows the identical pattern — a dedicated prompt, strict JSON out, parsed server-side — but the target shape is richer, because it has to match the app's existing `RoadmapContent` type exactly so the roadmap page can render it with no translation layer:

supabase/functions/ai-agent/index.ts

```
const ROADMAP_GENERATION_PROMPT = `You are a roadmap-generation assistant. You will
receive the transcript of a conversation between an AI roadmap agent and a user
discussing their learning goals. Extract or infer the details and respond with ONLY
a JSON object – no markdown, no code fences, no explanation outside the JSON – in
exactly this shape:
{
  "title": "<short roadmap title>",
  "goal": "<the user's stated goal in one sentence>",
  "current_level": "<assessed current skill level>",
  "target_level": "<the target level to reach>",
  "duration_weeks": <integer, total weeks>,
  "missing_skills": ["<skill1>", "<skill2>"],
  "weekly_plan": [
    { "week": 1, "title": "<week title>", "topics": ["<topic1>"],
      "goals": ["<goal1>"], "problems": <integer> }
  ],
  "resources": [
    { "title": "<resource title>", "type": "<video|article|practice|book>",
      "url": "<url or omit>", "description": "<one line>" }
  ]
}
Be specific with topics and weeks – cover the full duration_weeks with one weekly_plan
entry per week.`;
```

On the client, each returned `weekly_plan` entry gets a `completed: false` flag stapled on before the insert — that flag doesn't exist in the model's output at all, it's added purely so the Roadmap page has something to toggle:

```
await supabase.from('roadmaps').insert({
  title: r.title,
  goal: r.goal,
  content: {
    current_level: r.current_level,
    target_level: r.target_level,
    duration_weeks: r.duration_weeks,
    missing_skills: r.missing_skills,
    weekly_plan: r.weekly_plan.map(w => ({ ...w, completed: false })),
    resources: r.resources,
  },
})
```

Checking a week off later is a partial-merge update — read the roadmap's current `content`, flip one week's `completed` boolean, write the whole object back:

```
const toggleWeek = async (roadmap: Roadmap, weekIndex: number) => {
  const updatedWeeklyPlan = roadmap.content.weekly_plan.map((w, i) =>
    i === weekIndex ? { ...w, completed: !w.completed } : w
  )
  const updatedContent = { ...roadmap.content, weekly_plan: updatedWeeklyPlan }
  await supabase.from('roadmaps').update({ content: updatedContent }).eq('id', roadmap.id)
}
```


10 Client Integration

On the frontend, `sendMessage` in `ChatInterface.tsx` is the single function every chat turn flows through: create the chat row if this is the first message, save the user's message, call the edge function, save the assistant's reply, then — only for DSA turns that returned a topic — bump progress and tell the parent component to refresh the insights panel:

`src/components/workspace/ChatInterface.tsx`

```
const result = await response.json().catch(() => ({}))

if (!response.ok || result.error) {
  throw new Error(result.error || `Agent error: ${response.status}`)
}

const detectedAgent = (result.agent_type as AgentType) || currentAgentType

const { data: aiMsg } = await supabase
  .from('messages')
  .insert({
    chat_id: activeChatId,
    role: 'assistant',
    content: result.content,
    agent_type: detectedAgent,
    metadata: result.metadata ?? {},
  })
  .select()
  .single()

if (aiMsg) {
  setMessages(prev => [...prev, aiMsg])
  setAgentType(detectedAgent)
}

if (user && result.metadata?.topic) {
  await bumpTopicProgress(user.id, result.metadata.topic)
  onProgressUpdate?.()
}
```

`bumpTopicProgress` itself is intentionally silent on failure — progress tracking is a nice-to-have layered on top of the chat, not something that should ever be able to break the actual conversation if a database call hiccups:

```

async function bumpTopicProgress(userId: string, topic: string) {
  try {
    const { data: existing } = await supabase
      .from('progress_reports')
      .select('id, score, problems_solved')
      .eq('user_id', userId)
      .eq('topic', topic)
      .maybeSingle()

    if (existing) {
      await supabase
        .from('progress_reports')
        .update({
          score: Math.min(100, existing.score + 5),
          problems_solved: existing.problems_solved + 1,
        })
        .eq('id', existing.id)
    }
  } catch {
    // best-effort progress tracking; a failure here shouldn't break the chat
  }
}

```

`onProgressUpdate` is a callback threaded down from `Workspace.tsx`, which holds a `progressVersion` counter purely so the always-mounted AI Insights panel knows when to refetch — without it, the panel had no way to learn that a background update had happened and would keep showing stale numbers until a full page reload.

11 Security Notes

- **Every request is auth-checked in the function itself** (Section 3), not just at the gateway — because the gateway's JWT check can't be used at all with the newer publishable-key format.
- **Row-level security is the real backstop.** Even if a request somehow reached the database with the wrong user's ID, Postgres itself rejects the read/write — this was verified directly by attempting a cross-user read with no auth token and confirming it returned an empty result set, not an error and not someone else's data.
- **The Groq API key never reaches the client.** It's a Supabase secret, read only inside the edge function's Deno runtime; the browser only ever sees the response text.
- **Structured actions reuse the same auth gate.** `action: "end_interview"` and `action: "generate_roadmap"` are checked inside the same handler as regular chat, after the same `getUser(token)` call — there's no separate, easier-to-forget auth path for them.
- **Defensive JSON parsing assumes the model will occasionally misbehave.** `extractJSON` strips markdown fences and falls back to regex-extracting the first `{...}` block before giving up, because "the model always returns exactly what the prompt asked for" is not a safe assumption to build error handling around.